As of Windows 10, you can run Linux inside Windows using the Windows Subsystem for Linux. Running Python inside the WSL is slightly more complicated than running it directly from Windows, as you need to be inside the Linux environment, but it has the added benefit that most deployment scenarios are Unix based. By using the WSL, your development environment will likely be a closer match to your production environment, which tends to lessen the challenges experienced during deployment. Documentation on installing the WSL is available here: https://learn.microsoft.com/en-us/windows/wsl/install.

A.2 Virtual environments

When you install a third-party package in Python using `pip` or its equivalent, the tool downloads Python code from the *Python Package Index* (PyPI) and puts it in the site-packages directory. When you `import` a package in your code, Python looks in this directory as well as a few other places.

> **The Cheese Shop**
>
> Python isn't named after the snake but after the British comedy troupe Monty Python. When it came to building a site to manage Python packages, the coders called it the "Cheese Shop," after a famous Monty Python skit. Eventually, reason won out, and it was renamed the Python Package Index, but it is still affectionately referred to by its original moniker on occasion.

It is really handy to be able to choose from the over 500,000 freely available Python third-party libraries, but there is a complication. Packages change over time, and multiple versions are available. If you created a website using Django in 2021, you likely were using Django 3.2. In the following year Django 4.0 was released. You may not want to go back to your original project and upgrade it, as installing Django 4.0 overwrites Django 3.2, possibly breaking your original site.

If all that isn't messy enough, most packages have dependencies on other packages. In 2022, when you installed Django 4.0, you automatically got asgiref 3.4.1 and sqlparse 0.2.2. What if you also wanted to insall Apache Superset, the data visualization platform, which also uses sqlparse? How do you know that the version used by Django and the version used by Superset don't conflict?

The answer to all these questions is to use *virtual environments*. A Python virtual environment creates small copies of all the directories that Python requires in their own place and sets up a separate site-packages directory. The important stuff, like the interpreter itself, is linked to, so it doesn't take up too much disk space. Each virtual environment is self-contained, so you can install a package into an environment without affecting any other environment, including the default install location.

To use a virtual environment, you activate it. Activating an environment changes your shell so that the environment's link to the Python interpreter is first in your path, meaning it is the one your terminal finds when you run a script. It is best practice to